DOCUMENTACIÓN PARA NO MORIR EN EL INTENTO

Material Didáctico · Desarrollo de Videojuegos para el taller de Pira Academy

PREFAB 01 — PLAYER

1. Introducción

Este documento describe todos los parámetros configurables del prefab Player desde el Inspector de Unity. Cada componente tiene su propia sección con una tabla de referencia rápida.

> 💡 *Consejo:Modificá los parámetros de a uno a la vez y probá en Play Mode para observar el impacto de cada cambio.*

Cómo leer las tablas:

- Campo — nombre exacto del parámetro tal como aparece en el Inspector.
- Tipo — qué tipo de dato acepta (float, bool, LayerMask, referencia a objeto, etc.).
- Rango / Default — límites del slider o valor recomendado de partida.
- Descripción — qué hace ese parámetro y cómo afecta al juego.

Machete de los tipos de datos:

- **Bool:** Solo puede tomar 2 valores, Verdadero o Falso. Pulsar esa opción en el inspector la vuelve verdadera caso contrario es falso.
- **Float:** Número decimal / que puede tener coma. Ej: 3.13 , 2.15, 1.15
- **Int:** Numero entero Ej: 1,2,3,4,5.
- **String:** Cadena de texto. Ejemplo: “Este es el mejor taller del mundo”.
- **Enum:** Los Enum son en resumidas cuentas, tipos de datos con valores definidos finitos. Ej: un enum *DIA_DE_SEMANA*: solo tiene como valores posibles. Lunes, Martes, Miércoles, Jueves, Viernes.
- **Vector2:** Es un tipo de dato con 2 valores dentro. Por ejemplo para determinar la posición en pantalla en un juego 2D se debe especificar el eje X, eje Y.
- **Vector3:** Es un tipo de dato con 3 valores dentro. Por ejemplo para determinar la posición en pantalla en un juego 3D se debe especificar el eje X, eje Y, eje Z. (Fua viste el copy paste que meti del anterior pero le meti un 3?, felicidades aprendiste que es un vector).

2. Jerarquía del Prefab

El prefab Player está organizado de la siguiente manera en la ventana Hierarchy:

```
📦 Player                    ← GameObject raíz (física + lógica)
   ├── Rigidbody 2D
   ├── Capsule Collider 2D
   ├── Player Input
   ├── PlayerController
   ├── PlayerVFX
   ├── SquashAndStretch
   ├── PlayerAudio
```

```
    ├── PlayerInteractor
    ├── CameraShakeListener
    │
    ├── 👁️  View              ← Hijo: visual del personaje
    │    ├── Sprite Renderer
    │    └── Animator
    │
    ├── ✨ VFX                 ← Hijo: efectos de partículas
    │    └── Particle System (x3 configurados por código)
    │
    └── 🔵 InteractionPoint  ← Hijo: origen del radio de interacción
```

⚠️ *Nunca muevas la física (Rigidbody 2D, Collider) al hijo View. La física SIEMPRE vive en el objeto raíz. El hijo View solo contiene lo visual para que el squash/stretch no afecte al collider.*

3. Componentes Nativos de Unity

3.1 Rigidbody 2D

Es el motor de la física del personaje. Unity lo mueve automáticamente según las fuerzas que el código le aplica.

Parámetros relevantes de conocer (NO LO MODIFIQUES GRACIAS)

Campo	Tipo	Rango / Default	Descripción
Gravity Scale	Float	Default: 3–4	Qué tan fuerte cae el personaje. El código también modifica esto en tiempo real (escaleras, salto). Valor bajo = caída lenta/flotante; Valor alto = caída pesada.
Linear Drag	Float	Default: 0	Fricción con el aire. En un platformer generalmente se deja en 0 y el frenado se maneja desde PlayerController.
Collision Detection	Enum	Continuous	Continuous previene que el jugador 'atraviese' plataformas a alta velocidad. No cambiar a Discrete.
Freeze Rotation Z	Bool	✅ Activado	Impide que el personaje rote al chocar. Debe estar siempre activado.

3.2 Capsule Collider 2D

Define la forma física del jugador. Se utilizan cápsulas comúnmente porque desliza suavemente por esquinas sin quedarse trabado.

Parámetros relevantes de conocer (Solo modificar el Size si quieren ajustar)

Campo	Tipo	Rango / Default	Descripción
Size (X, Y)	Vector2	Ajustar al sprite	Ancho y alto de la cápsula. Debería cubrir el cuerpo del sprite sin incluir espacios vacíos del asset.
Direction	Enum	Vertical	Orientación de la cápsula. Para un personaje erguido siempre debe ser Vertical.
Material	PhysicsMaterial2D	NoFriction (auto)	El código crea automáticamente un material sin fricción. No asignar un material manual salvo que sea intencional.

3.3 Player Input

Componente de Unity que conecta el Input Actions Asset (teclado, gamepad, táctil) con los métodos del código.

Parámetros relevantes de conocer (NO LO MODIFIQUES GRACIAS)

Campo	Tipo	Rango / Default	Descripción
Actions	InputAction Asset	Requerido	Archivo .inputactions que define las acciones del juego (Move, Jump, Interact, etc.). Arrastrá el asset desde el Project.
Behavior	Enum	Send Messages	Cómo se comunica con los scripts. Debe coincidir con el estilo usado en PlayerController (OnMove, OnJump) y PlayerInteractor (OnInteract).
Default Map	String	Player	El Action Map activo al inicio. Cambiar si tenés múltiples mapas (ej: "UI" para menús).

4. Scripts Personalizados

4.1 PlayerController

El cerebro del jugador. Gestiona el movimiento, el salto, las escaleras, las plataformas semisólidas y el sistema de daño/knockback. Todos los demás scripts reaccionan a los eventos que este emite.

📡 *Este script emite 3 eventos que otros componentes escuchan: OnJumpPerformed, OnLandPerformed y OnHitPerformed (estático). Pensalo como una radio: PlayerController transmite, y VFX/Audio/Squash son los receptores.*

🏃 Movimiento

Campo	Tipo	Rango / Default	Descripción
Move Speed	Float	1 – 20 | 8	Velocidad horizontal máxima en unidades/segundo. Valores bajos = sensación pesada; altos = personaje muy veloz.
Movement Smoothing	Float	0 – 0.2 | 0.05	Suavizado de la aceleración. 0 = respuesta instantánea; 0.2 = arranque muy gradual. Para un platformer ágil usar valores cercanos a 0.
Dash Burst	Float	0 – 10 | 5	Impulso extra al iniciar el movimiento o cambiar de dirección. Da sensación de peso y snappiness al arranque.

🦘 Salto

Campo	Tipo	Rango / Default	Descripción
Jump Force	Float	5 – 30 | 16	Fuerza inicial del salto en unidades. Directamente proporcional a la altura máxima alcanzada.
Coyote Time	Float	0 – 0.5 | 0.15	Segundos de gracia para saltar después de caer de una plataforma. Hace que el control sea más forgiving sin notarse artificialmente.
Jump Buffer Time	Float	0 – 0.5 | 0.1	Segundos que se guarda en memoria una pulsación de salto antes de aterrizar. Si el jugador pulsa justo antes de tocar el suelo, igual salta.
Fall Extra Gravity	Float	1 – 5 | 2.5	Multiplicador de gravedad extra al caer. Valores altos = caída más rápida y pesada (feel arcade). 1 = gravedad normal.
Low Jump Extra Gravity	Float	1 – 5 | 2	Gravedad extra si se suelta el botón de salto antes de llegar al pico. Permite saltos cortos y largos con el mismo botón.

Escaleras (No tengo emoji pa esto en google docs todo mal)

Campo	Tipo	Rango / Default	Descripción
Ladder Layer	LayerMask	Capa personalizada	Qué objetos son tratados como escalera. Asigná la capa que usaste para las escaleras en tu proyecto.
Climb Speed	Float	1 – 10 | 5	Velocidad de movimiento vertical al trepar.

⬇️ One-Way Platforms (Plataformas de la que podés dejarte caer)

Campo	Tipo	Rango / Default	Descripción
One Way Platform Layer	LayerMask	Capa personalizada	Capa que identifica plataformas que se pueden atravesar hacia abajo (Down + Jump).
Drop Down Duration	Float	Default: 0.5	Segundos que dura el estado de caída a través de la plataforma antes de volver a colisionar con ella.

⚔️ Combate y Daño

Campo	Tipo	Rango / Default	Descripción
Damage Layer	LayerMask	Capa de enemigos	Qué capas pueden dañar al jugador al colisionar o entrar en trigger.
Damage Cooldown	Float	0 – 2 | 1	Segundos de invulnerabilidad tras recibir un golpe. Evita recibir daño múltiple en el mismo frame.
Knockback Force	Vector2	Default: (10, 8)	Fuerza de empuje al recibir daño. X = empuje horizontal (aleja del golpe); Y = rebote vertical.
Hit Stun Duration	Float	Default: 0.3	Segundos que el jugador queda sin control después de ser golpeado.

🟩 Detección de Suelo

Campo	Tipo	Rango / Default	Descripción
What Is Ground	LayerMask	Capa de suelo	Qué capas se consideran suelo para la detección de aterrizaje.
Ground Check	Transform	Referencia	Objeto vacío hijo del Player, posicionado en los pies del personaje. Define el centro del box de detección.

Campo	Tipo	Rango / Default	Descripción
Ground Check Size	Vector2	Default: (0.8, 0.1)	Tamaño del rectángulo invisible de detección de suelo. X debe ser ligeramente menor que el collider del jugador para evitar falsos positivos en paredes.

🔧 Debug

Campo	Tipo	Rango / Default	Descripción
Show Debug Gizmos	Bool	Default: ✅	Muestra en la Scene View un rectángulo verde/rojo en los pies del jugador indicando si está en suelo. Solo visible en el editor, no en el build final.

4.2 PlayerVFX

Gestiona los efectos de partículas del jugador. Se suscribe a los eventos de PlayerController para disparar cada efecto en el momento exacto.

> ℹ️ *Los 3 Particle Systems son configurados automáticamente por código en Awake (simulación en espacio mundo, sin loop automático). Solo necesitás asignar las referencias y ajustar cantidades desde el Inspector como mejor te parezca.*

📋 Referencias

Campo	Tipo	Rango / Default	Descripción
Player Controller	PlayerController	Requerido	Referencia al componente PlayerController del mismo objeto. Sin esta referencia no se suscribirá a ningún evento.
Jump Dust	ParticleSystem	Requerido	Sistema de partículas que se emite al saltar.
Land Dust	ParticleSystem	Requerido	Sistema de partículas que se emite al aterrizar.
Hit Effect	ParticleSystem	Requerido	Sistema de partículas que se emite al recibir un golpe.

💨 Configuración de Emisión

Campo	Tipo	Rango / Default	Descripción
Jump Particles Count	Int	Default: 6	Cantidad de partículas emitidas en cada salto.
Land Particles Count	Int	Default: 10	Cantidad de partículas emitidas al aterrizar. Más cantidad = aterrizaje más impactante.
Hit Particles Count	Int	Default: 15	Cantidad de partículas emitidas al recibir daño.

📍 Offsets de Posición

Campo	Tipo	Rango / Default	Descripción
Foot Offset	Vector3	Default: (0, -0.5, 0)	Desplazamiento relativo al centro del Player donde aparecen las partículas de salto y aterrizaje. Ajustar según la altura del sprite.
Center Offset	Vector3	Default: (0, 0, 0)	Posición del efecto de golpe. Por defecto en el centro del personaje.

4.3 SquashAndStretch

Aplica deformación visual (aplastamiento y estiramiento) al sprite cuando el jugador salta y aterriza. Es una técnica clásica de animación que da vida y peso al personaje sin necesitar fotogramas extra.

🎨 *Analogía: Imaginá una pelota de goma. Al saltar se estira (alargada), al caer se aplana (achatada) y luego vuelve a su forma normal. Eso es exactamente lo que hace este componente.*

📋 Referencias

Campo	Tipo	Rango / Default	Descripción
Player Controller	PlayerController	Requerido	Fuente de los eventos de salto y aterrizaje.
Visuals Root	Transform	Hijo View	El Transform que será deformado. DEBE ser el objeto hijo con el SpriteRenderer (View), nunca el objeto raíz con la física.

🔧 Configuración

Campo	Tipo	Rango / Default	Descripción
Jump Stretch	Vector2	Default: (0.75, 1.25)	Factor de escala al saltar. X < 1 = más angosto; Y > 1 = más alto. Mantené X*Y ≈ 1 para conservar el área visual.
Land Squash	Vector2	Default: (1.25, 0.75)	Factor de escala al aterrizar. Inverso al Stretch: más ancho y menos alto.
Return Speed	Float	Default: 15	Velocidad de regreso a la escala normal (1,1). Valor alto = rebote rápido; valor bajo = deformación lenta y exagerada.

4.4 PlayerAudio

Reproduce efectos de sonido en respuesta a los eventos del jugador. Utiliza un sistema de canales (AudioEventChannelSO) para desacoplar la reproducción de audio del código del jugador.

> ⚠️ *Es un quilombo para que lo entienda un principiante, pero vos solo lee los parámetros si quieres cambiar algún sonido.*

🎵 Audio Configs

Campo	Tipo	Rango / Default	Descripción
Jump SFX	AudioConfigSO	Requerido	ScriptableObject con la configuración del sonido de salto (clip, volumen, pitch aleatorio, etc.).
Land SFX	AudioConfigSO	Requerido	ScriptableObject con la configuración del sonido de aterrizaje.
Hit SFX	AudioConfigSO	Requerido	ScriptableObject con la configuración del sonido al recibir daño.

4.5 PlayerInteractor

Detecta objetos interactuables en el entorno y ejecuta su lógica al presionar el botón de interacción.

⚙️ Interaction Settings

Campo	Tipo	Rango / Default	Descripción
Interaction Radius	Float	Default: 1.5	Radio del círculo de detección de interacción en unidades de Unity. Un gizmo amarillo lo visualiza en la Scene View al seleccionar el objeto.
Interaction Point	Transform	Recomendado: hijo InteractionPoint	Centro del círculo de detección. Por defecto usa el transform del propio jugador, pero es mejor asignar el hijo InteractionPoint para tener control preciso de la posición (ej: frente al personaje).
Interactable Layer	LayerMask	Capa personalizada	Solo detecta objetos en esta capa. Asigná la capa que uses para todos los objetos interactuables de tu juego (NPCs, cofres, palancas, etc.).

4.6 CameraShakeListener

Componente mínimo que expone la posición de la cámara como una propiedad estática accesible desde cualquier script sin necesidad de guardar una referencia directa.

> ℹ️ *No tiene parámetros configurables en el Inspector. Simplemente se agrega al GameObject de la cámara (o al Player si es cámara fija) y otros sistemas pueden leer CameraShakeListener.Position para saber dónde está la cámara en cada frame.*

5. GameObjects Hijos

5.1 View (Sprite Renderer + Animator)

Contiene todo lo visual del personaje. Está separado del objeto raíz para que las deformaciones de SquashAndStretch no afecten al Collider ni al Rigidbody.

🖼️ Sprite Renderer

Campo	Tipo	Rango / Default	Descripción
Sprite	Sprite	Sprite del personaje	El gráfico estático del personaje. Si usás animaciones, el Animator lo sobreescribe en cada frame.
Sorting Layer	SortingLayer	Player (recomendado)	Capa de orden de renderizado. Asegurate de que el jugador se vea por encima del fondo pero detrás de los elementos de UI.
Order in Layer	Int	0	Orden dentro del Sorting Layer. Útil si hay múltiples objetos en la misma capa.
Flip X / Flip Y	Bool	No tocar	El script PlayerController gestiona el flip horizontal a través del LocalScale del objeto raíz. No cambiar este valor manualmente.

🎬 Animator (No hace falta tocarlo)

Campo	Tipo	Rango / Default	Descripción
Controller	AnimatorController	Requerido	El Animator Controller que define los estados de animación (idle, run, jump, fall, climb, hit, duck). PlayerController actualiza sus parámetros automáticamente.

Campo	Tipo	Rango / Default	Descripción
Avatar	Avatar	None (2D)	En proyectos 2D se deja vacío.
Update Mode	Enum	Normal	Normal actualiza la animación en sync con Update. No cambiar salvo necesidad específica.

📝 *Parámetros del Animator que maneja el código automáticamente: Speed (Float), IsGrounded (Bool), VerticalVelocity (Float), IsClimbing (Bool), ClimbSpeed (Float), IsDucking (Bool), Hit (Trigger), IsHit (Bool). No necesitás configurarlos a mano,ya está todo hecho.*

5.2 VFX (Particle Systems)

Contiene los 3 sistemas de partículas usados por PlayerVFX. El script los configura automáticamente en Awake, pero podés ajustar la apariencia visual desde el Inspector de cada Particle System.

⚙️ *El código fuerza estas configuraciones en todos los PS al inicio: Simulation Space = World, Loop = false, Play On Awake = false, Rate Over Time = 0. Cualquier cambio a estas propiedades desde el Inspector será ignorado/sobreescrito en runtime.*

✨ Qué sí podés personalizar libremente

Campo	Tipo	Rango / Default	Descripción
Start Lifetime	Float / Curve	Módulo Main	Cuánto tiempo viven las partículas. Más tiempo = efecto más prolongado.
Start Speed	Float / Curve	Módulo Main	Velocidad inicial de las partículas al ser emitidas.
Start Size	Float / Curve	Módulo Main	Tamaño inicial de cada partícula.
Start Color	Color / Gradient	Módulo Main	Color o gradiente de color inicial.
Gravity Modifier	Float	Módulo Main	Cuánto afecta la gravedad a las partículas. 0 = flotan; 1 = caen como el jugador.
Shape	Módulo Shape	Cone, Circle, etc.	Forma desde la que se emiten las partículas. Un Cone ancho para polvo de aterrizaje; un Circle para efecto de golpe.
Color Over Lifetime	Gradient	Módulo opcional	Permite que las partículas se desvanezcan al final de su vida (recomendado activar).
Size Over Lifetime	Curve	Módulo opcional	Permite que las partículas encojan o crezcan durante su vida.

5.3 InteractionPoint

Un GameObject vacío sin componentes. Solo sirve como punto de referencia de posición para PlayerInteractor.

Campo	Tipo	Rango / Default	Descripción
Position (Transform)	Vector3	Local al Player	Ajustar para que quede frente al personaje (ej: X = 0.5 si mira a la derecha). PlayerInteractor usa esta posición como centro del radio de interacción.

6. Referencia Rápida — Problemas Comunes

Síntoma	Qué revisar
El jugador cae muy rápido / muy lento	Ajustar Gravity Scale en Rigidbody 2D y/o Fall Extra Gravity en PlayerController.
El salto no se registra bien	Aumentar Jump Buffer Time. Si cae de plataformas y no puede saltar, verificar Coyote Time.
No aparecen partículas	Revisar las referencias de Jump Dust / Land Dust / Hit Effect en PlayerVFX y que playerController esté asignado.
No se escucha el audio	Verificar que el PlayerAudio tenga las referencias a las audio configs.Y esas referencias tengan sonido asignado
No funciona la interacción	Confirmar que el objeto interactuable está en la capa de Interactable Layer y tiene un componente con IInteractable.
El jugador no salta o se traba su animación	Si lo que dibujaste como suelo no está en el layer Ground, no va a considerar que el player esté en el suelo.

Prefab 01 — Player · Continúa en la siguiente sección

PREFABS 02 & 03 — BEE / SLIME

1. Introducción

Bee y Slime son enemigos de patrulla simples que comparten exactamente la misma estructura de prefab y los mismos componentes. La única diferencia entre ambos es el arte (sprite y animaciones). Todo lo que se documenta aquí aplica igualmente a los dos prefabs.

🔁 *Comparten componentes: Rigidbody 2D, Box Collider 2D y SimplePatrolEnemy. El comportamiento se define una sola vez en el script y se reutiliza en ambos prefabs cambiando solo las referencias visuales.*

2. Jerarquía del Prefab

Estructura en la ventana Hierarchy (idéntica para Bee y Slime):

```
📦 Bee / Slime              ← GameObject raíz (física + lógica)
   ├── Rigidbody 2D
   ├── Box Collider 2D
   ├── SimplePatrolEnemy
   │
   ├── 👁️  View             ← Hijo: visual del personaje
   │    ├── Sprite Renderer
   │    └── Animator
   │
   ├── 🔵 PointA            ← Hijo: extremo izquierdo de la patrulla
   └── 🔵 PointB            ← Hijo: extremo derecho de la patrulla
```

📏 *Analogía: PointA y PointB funcionan como los dos extremos de un péndulo. El enemigo oscila de un punto al otro indefinidamente. Para definir el rango de patrulla, simplemente movés estos hijos en la Scene View.*

3. Componentes Nativos de Unity

3.1 Rigidbody 2D

Gestiona la física del enemigo. A diferencia del Player, el código lo configura en modo Kinematic automáticamente: no responde a la gravedad ni a fuerzas externas, solo se mueve mediante código.

> ℹ️ *Body Type se fuerza a Kinematic por código en Awake. No hace falta cambiarlo desde el Inspector; si lo cambiás manualmente será sobreescrito al iniciar el juego.*

Parámetros relevantes de conocer (NO LO MODIFIQUES GRACIAS)

Campo	Tipo	Rango / Default	Descripción
Collision Detection	Enum	Continuous	Mantener en Continuous para que el enemigo detecte colisiones correctamente a altas velocidades.
Freeze Rotation Z	Bool	✅ Activado	Impide que el enemigo rote al chocar con paredes o el jugador. Siempre debe estar activado.

3.2 Box Collider 2D

Define la forma física del enemigo para colisiones y detección de daño. Se usa Box en lugar de Capsule porque la mayoría de enemigos sprite tienen siluetas más rectangulares.

Parámetros relevantes de conocer (Solo modificar el Size si quieren ajustar)

Campo	Tipo	Rango / Default	Descripción
Size (X, Y)	Vector2	Ajustar al sprite	Ancho y alto del collider. Ajustarlo para que cubra el cuerpo del sprite sin exceder sus bordes. Un collider más pequeño que el sprite suele sentirse más justo para el jugador.
Offset (X, Y)	Vector2	Centrado en el sprite	Desplazamiento del collider respecto al pivot del objeto. Útil si el sprite no está perfectamente centrado en el pivot.
Is Trigger	Bool	❌ Desactivado	Si se activa, el enemigo deja de bloquear físicamente al jugador y solo detecta

Campo	Tipo	Rango / Default	Descripción
			solapamiento. Desactivado para enemigos con colisión sólida.

4. Script — SimplePatrolEnemy

Controla todo el comportamiento del enemigo: movimiento de patrulla entre dos puntos, pausa en los extremos y el efecto visual de squash & stretch al girar.

> ⚙️ *El enemigo no tiene sistema de daño propio: daña al jugador porque su GameObject está en la capa que PlayerController tiene configurada como Damage Layer. No hace falta código extra en el enemigo para eso.*

📍 Configuración de Ruta

Campo	Tipo	Rango / Default	Descripción
Point A	Transform	Requerido (hijo PointA)	Extremo inicial de la patrulla. Arrastrá el hijo PointA desde la jerarquía. Si se deja vacío, el script se desactiva con un error en consola.
Point B	Transform	Requerido (hijo PointB)	Extremo final de la patrulla. Arrastrá el hijo PointB desde la jerarquía.

> 💡 *Los puntos se leen solo una vez en Start y se guardan como Vector3. Mover PointA/PointB en runtime (fuera del editor) no afecta la ruta activa. Para rutas dinámicas se necesitaría un sistema diferente.*

🏃 Movimiento

Campo	Tipo	Rango / Default	Descripción
Speed	Float	0 – 10 | 3	Velocidad de patrulla en unidades/segundo. Solo afecta al eje X; el enemigo no se mueve verticalmente (Kinematic sin gravedad).
Wait Time At Edge	Float	0 – 5 | 1	Segundos que el enemigo espera al llegar a cada extremo antes de girar. 0 = giro instantáneo sin pausa.
Threshold	Float	Default: 0.2	Distancia mínima en unidades al punto destino para considerarlo como alcanzado y activar el giro. Valores muy bajos pueden

Campo	Tipo	Rango / Default	Descripción
			causar overshooting en enemigos rápidos; valores muy altos hacen que gire antes de llegar al punto.

🎨 Squash & Stretch

El efecto de squash & stretch ocurre en el momento del giro (cuando el enemigo llega a un extremo y hace pausa). Le da vida y peso al personaje sin necesitar fotogramas de animación adicionales.

Campo	Tipo	Rango / Default	Descripción
Squash X	Float	0.3 – 0.9 | 0.6	Factor de aplastamiento horizontal al girar. 0.6 = el sprite se comprime al 60% de su ancho. Menor valor = squash más exagerado.
Stretch Y	Float	1.1 – 2.0 | 1.3	Factor de estiramiento vertical al girar. 1.3 = el sprite se alarga al 130% de su alto. Mayor valor = stretch más exagerado.
Squash Duration	Float	0.05 – 0.5 | 0.15	Duración total en segundos de la animación completa (deformación + retorno). El script divide este tiempo en dos mitades iguales: mitad para deformar, mitad para volver a la escala normal.

⚠️ *Squash X y Stretch Y afectan al objeto raíz del enemigo directamente (no a un hijo View como en el Player). Esto está bien porque el enemigo no tiene física separada de lo visual en este prefab. Si hubiéramos optado por separar el sprite a un hijo, este script debería apuntar a ese hijo como hicimos en Player.*

5. GameObjects Hijos

5.1 View (Sprite Renderer + Animator)

Igual que en el Player, contiene el arte del personaje. En estos enemigos simples el Animator puede usarse para una animación de idle/walk en loop.

🖼️ Sprite Renderer

Campo	Tipo	Rango / Default	Descripción
Sprite	Sprite	Sprite del enemigo	El gráfico del enemigo. Si usás Animator, este campo es sobreescrito por la animación en cada frame.
Sorting Layer	SortingLayer	Enemies (recomendado)	Capa de renderizado. Asegurate de que los enemigos se vean por encima del fondo pero en el orden correcto respecto al jugador.
Order in Layer	Int	0	Orden dentro del Sorting Layer. Ajustar si dos enemigos en la misma capa se superponen incorrectamente.

🎬 Animator

Campo	Tipo	Rango / Default	Descripción
Controller	AnimatorController	Requerido	El Animator Controller del enemigo. Generalmente contiene un estado de Walk en loop. SimplePatrolEnemy no escribe ningún parámetro en el Animator (el movimiento lo maneja el Rigidbody), pero podés agregar parámetros propios si lo extendés.

5.2 PointA y PointB

GameObjects vacíos que definen los extremos de la ruta de patrulla. Su única propiedad relevante es la posición en el mundo.

Campo	Tipo	Rango / Default	Descripción
Position X (Transform)	Float	Posición en el mundo	Coordenada horizontal del punto de patrulla. La posición Y no importa ya que el enemigo solo se mueve en el eje X.
Position Y (Transform)	Float	Ignorado en runtime	El script solo usa la coordenada X de estos puntos. Podés dejar Y en 0 o alineado con el suelo; no afecta al movimiento.

📐 *Flujo de trabajo recomendado en el Editor: Seleccioná el prefab en la Scene View → mové el hijo PointA al extremo izquierdo deseado → mové PointB al extremo derecho. El gizmo amarillo en la Scene View mostrará la línea de patrulla en tiempo real.*

6. Gizmos en la Scene View

SimplePatrolEnemy dibuja automáticamente ayudas visuales en el editor:

Gizmo	Qué indica
Línea amarilla	Conecta PointA con PointB. Muestra la ruta completa de patrulla de un vistazo.
Esferas amarillas	Marcan la posición exacta de cada punto de giro (PointA y PointB).

7. Referencia Rápida — Problemas Comunes

Síntoma	Qué revisar
El enemigo no se mueve	Verificar que PointA y PointB estén asignados. Si hay un error en consola '[SimplePatrolEnemy] Faltan puntos de ruta', el script se desactivó automáticamente.
El enemigo no gira en el extremo	Aumentar Threshold. Si la velocidad es alta el enemigo puede pasar de largo sin que se detecte que llegó al punto.
El enemigo no daña al jugador	Verificar que el GameObject del enemigo esté en la capa configurada como Damage Layer en el PlayerController del jugador.
El squash/stretch se ve extraño	Revisar que Squash X < 1 y Stretch Y > 1. Si los valores son similares entre sí no se notará el efecto. Intentá valores más extremos como (0.5, 1.5).
El enemigo cae por las plataformas	El Rigidbody está en modo Kinematic: no tiene gravedad. Si el enemigo cae, probablemente otro sistema esté modificando su posición Y, o hay un conflicto con otro Rigidbody.

Prefabs 02 & 03 — Bee / Slime · Continúa en la siguiente sección

PREFABS 04 · 05 · 06 — TRAMPOLINE / BOMB / LEVER

Prefab 04 — Trampoline

El Trampoline es un objeto de escenario que lanza al jugador hacia arriba al pisarlo. No tiene física propia; actúa como un trigger que llama al método Bounce del PlayerController.

Jerarquía

```
📦 Trampoline                ← GameObject raíz
   ├── Sprite Renderer
   ├── Box Collider 2D       (isTrigger = true, forzado por código)
   └── Trampoline.cs
```

Sprite Renderer

Renderiza el sprite del trampolín. Trampoline.cs controla manualmente qué sprite se muestra (idle vs. activo) según si fue pisado recientemente.

Parámetros relevantes

Campo	Tipo	Rango / Default	Descripción
Sprite	Sprite	Sprite idle inicial	El gráfico en reposo. El script lo sobreescribe al activarse; se restaura automáticamente luego del Active Duration.
Sorting Layer / Order	SortingLayer / Int	Props o similar	Capa de renderizado. Ajustar para que el trampolín se vea por encima del suelo pero detrás del jugador si corresponde.

Box Collider 2D

El código fuerza isTrigger = true en Awake. El trampolín detecta al jugador por solapamiento, no por colisión física. No hace falta configurar el trigger manualmente.

Parámetros relevantes

Campo	Tipo	Rango / Default	Descripción
Size (X, Y)	Vector2	Ajustar al sprite	El área de activación. Si es muy pequeña el jugador no lo activa fácilmente; si es muy grande puede activarse sin pisar el trampolín visualmente.
Offset	Vector2	Centrado	Desplazamiento del trigger. Útil si el trampolín tiene un área de rebote más estrecha que su sprite completo.

Script — Trampoline

⚙️ Configuración Mecánica

Campo	Tipo	Rango / Default	Descripción
Bounce Force	Float	Default: 25	Fuerza de impulso vertical que se le aplica al jugador al rebotar. Se pasa directamente al método Bounce() del PlayerController. Valores altos = salto muy alto; demasiado alto puede sacar al jugador fuera del nivel.
Player Layer	LayerMask	Capa del jugador	Solo objetos en esta capa activan el rebote. Asignar la capa donde está el Player para evitar que enemigos u otros objetos activen el trampolín.

🎨 Configuración Visual

Campo	Tipo	Rango / Default	Descripción
Idle Sprite	Sprite	Sprite en reposo	Sprite mostrado por defecto cuando el trampolín no ha sido activado recientemente.
Active Sprite	Sprite	Sprite comprimido	Sprite mostrado en el momento del rebote para dar feedback visual de activación (generalmente el trampolín aplastado).
Active Duration	Float	Default: 0.2	Segundos que se mantiene el Active Sprite antes de volver al Idle Sprite. Muy corto = apenas se nota; 0.2–0.3s es suficiente para dar feedback claro.

💥 Feedback

Campo	Tipo	Rango / Default	Descripción
Bounce VFX Prefab	GameObject	Opcional	Prefab de partículas o efecto que se instancia en la posición del trampolín al rebotar. Dejar vacío para deshabilitar el VFX.

⚠️ *Warning de rendimiento, no lo usen de ejemplo: El VFX se instancia con Instantiate() cada vez que se activa el trampolín. Si el jugador lo pisa repetidamente pueden acumularse objetos en la escena. Para niveles con uso intensivo del trampolín, habria que considerar usar un sistema de pooling para el VFX.*

Prefab 05 — Bomb

La bomba es un proyectil que cae desde el cielo, detona al tocar el suelo con un contador de mecha, y genera una onda de choque (ShockwaveVFX) junto con screen shake. Usa Object Pooling gestionado por el BombSpawner; normalmente no se coloca directamente en la escena.

> 🔄 *Object Pool: Las bombas nunca se destruyen realmente (salvo exceder el maxSize del pool). Al explotar se desactivan y vuelven al pool del BombSpawner para ser reutilizadas. Esto evita garbage collection frecuente.*

Jerarquía

```
📦 Bomb                      ← GameObject raíz (física + lógica)
   ├── Sprite Renderer
   ├── Circle Collider 2D    (colisión con suelo)
   ├── Rigidbody 2D          (gravedad activa, cae libremente)
   └── Bomb.cs
   │
   └── ✨ VFX                ← Hijo: onda de choque
        ├── Sprite Renderer  (sprite circular de la onda)
        ├── Circle Collider 2D (detección de daño de la explosión)
        └── ShockwaveVFX.cs
```

Componentes Nativos — Raíz

Sprite Renderer

Muestra el sprite de la bomba. Bomb.cs lo controla para el efecto de parpadeo (alterna entre sprite normal y de advertencia durante la mecha).

Campo	Tipo	Rango / Default	Descripción
Sorting Layer / Order	SortingLayer / Int	Projectiles o similar	Ajustar para que la bomba se vea por encima del fondo. El componente lo deshabilita temporalmente en el momento de la explosión.

Circle Collider 2D

Detecta el impacto con el suelo para iniciar la mecha. No es trigger.

Campo	Tipo	Rango / Default	Descripción
Radius	Float	Ajustar al sprite	Radio de la cápsula de colisión. Ajustar para que coincida con el área visual de la bomba.

Rigidbody 2D

Permite que la bomba caiga con gravedad real desde el punto de spawn. A diferencia de los enemigos, aquí no es Kinematic.

Campo	Tipo	Rango / Default	Descripción
Gravity Scale	Float	Default: 1–3	Qué tan rápido cae la bomba. Valores más altos hacen que la caída sea más dramática e impredecible.
Collision Detection	Enum	Continuous	Mantener en Continuous para detectar el impacto con el suelo incluso a alta velocidad de caída.
Freeze Rotation Z	Bool	Opcional	Activar si no se desea que la bomba rote al rebotar contra superficies.

Script — Bomb

🎨 Visual Feedback

Campo	Tipo	Rango / Default	Descripción
Sprite Renderer	SpriteRenderer	Referencia (auto)	Referencia al SpriteRenderer. El script lo busca automáticamente en Awake si se deja vacío.
Default Sprite	Sprite	Sprite normal	Sprite de la bomba en estado normal (antes de activarse y durante el parpadeo en frames pares).
Warning Sprite	Sprite	Sprite de alerta	Sprite alternativo (típicamente con un signo de exclamación o color rojo) que se muestra durante el parpadeo de mecha.
Blink Interval	Float	Default: 0.2	Segundos entre cada cambio de sprite durante la mecha. Valores más bajos = parpadeo más frenético que comunica urgencia al jugador.

🧨 Fuse Settings (Mecha)

Campo	Tipo	Rango / Default	Descripción
Fuse Time	Float	Default: 2	Segundos desde el aterrizaje hasta la detonación. Este es el tiempo que tiene el jugador para alejarse. Valores bajos = más difícil y urgente.
Ground Layer	LayerMask	Capa de suelo	Qué capas activan la mecha al colisionar. Asignar la misma capa de suelo que usa el resto del juego.

📐 Spawn Animation

Campo	Tipo	Rango / Default	Descripción
Spawn Curve	Animation Curve	Ease out recomendado	Curva que controla cómo escala la bomba desde 0 hasta su tamaño normal al aparecer. Una curva con overshoot (escala > 1 y vuelve a 1) da un efecto de rebote elástico al aparecer.
Spawn Duration	Float	Default: 0.35	Duración en segundos de la animación de aparición.

💥 Explosion FX

Campo	Tipo	Rango / Default	Descripción
Shockwave VFX	ShockwaveVFX	Referencia al hijo VFX	Referencia al componente ShockwaveVFX del hijo. Arrastrá el hijo VFX desde la jerarquía. Si se deja vacío, la bomba detona sin efecto visual de onda.

📳 Screen Shake

Campo	Tipo	Rango / Default	Descripción
Shake Duration	Float	Default: 0.3	Cuántos segundos dura el temblor de cámara al explotar.
Shake Magnitude	Float	Default: 0.25	Intensidad del temblor. Valores altos pueden ser molestos; 0.1–0.3 es suficiente para impacto sin incomodar.

Hijo — VFX (ShockwaveVFX)

El hijo VFX es la onda de choque visual que se expande desde el centro de la explosión. No es un Particle System: es un sprite circular que se escala y desvanece por código.

🔵 *Analogía: Imaginá una piedra cayendo en el agua. El VFX es ese aro que se expande y se desvanece. El Circle Collider del hijo crece con él y puede usarse para detectar si el jugador fue alcanzado por la onda.*

Componentes del hijo VFX

Campo	Tipo	Rango / Default	Descripción
Sprite Renderer	SpriteRenderer	Sprite circular	Sprite de la onda (típicamente un círculo o anillo semi-transparente). El script controla su color y alpha.
Circle Collider 2D	CircleCollider2D	isTrigger = true	Collider que crece con la onda. Puede usarse para aplicar daño al jugador si está en el radio de la explosión. El script lo desactiva cuando la onda termina.

Script — ShockwaveVFX

Campo	Tipo	Rango / Default	Descripción
Max Scale	Float	Default: 6	Escala máxima a la que llega la onda antes de desvanecerse. Define el radio visual de la explosión en unidades de Unity.
Duration	Float	Default: 0.35	Segundos que tarda la onda en expandirse y desaparecer completamente.
Start Color	Color	Default: naranja semitransparente	Color inicial de la onda. El alpha va de este valor hasta 0 durante la animación. Cambiar para explosiones de hielo (azul), electricidad (amarillo), etc.

Prefab 06 — Lever

La palanca es un objeto interactuable que activa o desactiva el spawneo periódico de bombas. Implementa la interfaz IInteractable, por lo que el PlayerInteractor la detecta y llama a Interact() al presionar el botón de interacción.

Jerarquía

```
📦 Lever                    ← GameObject raíz
   ├── Sprite Renderer      (sprite de la palanca)
   ├── Box Collider 2D      (trigger)
   └── BombSpawner.cs
   │
   ├── 💬 VFX               ← Hijo: ícono flotante de feedback
   │     └── Sprite Renderer
   │
   ├── 📍 SpawnPoint        ← Hijo: posición de aparición de bombas
   │
   └── 🔵 RangeDetector     ← Hijo: zona de interacción
         ├── Circle Collider 2D  (isTrigger = true)
         └── RangeDetector.cs
```

Componentes Nativos — Raíz

Sprite Renderer

Muestra el sprite de la palanca. BombSpawner lo controla para mostrar el estado ON u OFF.

Box Collider 2D

Colisión física de la palanca como objeto de escenario.

Campo	Tipo	Rango / Default	Descripción
Size (X, Y)	Vector2	Ajustar al sprite	El área física de la palanca. Ajustar para que coincida con la parte sólida del sprite.

Script — BombSpawner

🏭 *Arquitectura: BombSpawner usa Object Pooling interno. Al iniciarse, crea un pool de bombas vacío. Cada vez que spawnea, saca una bomba del pool (o crea una nueva si está vacío) en lugar de hacer Instantiate en cada ciclo. Las bombas se devuelven al pool al explotar.*

🎮 Lever Settings

Campo	Tipo	Rango / Default	Descripción
Lever Renderer	SpriteRenderer	Requerido	Referencia al Sprite Renderer del objeto raíz. Arrastrá el componente del raíz aquí.
Lever On Sprite	Sprite	Sprite palanca ON	Sprite que muestra la palanca en posición activa (bombas spawneando).
Lever Off Sprite	Sprite	Sprite palanca OFF	Sprite que muestra la palanca en posición inactiva.
Is On	Bool	Default: ❌ false	Estado inicial de la palanca al comenzar la escena. Activar si querés que el spawner empiece funcionando sin que el jugador la active.

💣 Spawn Settings

Campo	Tipo	Rango / Default	Descripción
Range Detector	RangeDetector	Referencia al hijo	Referencia al componente RangeDetector del hijo homónimo. Si se asigna, el jugador solo puede interactuar con la palanca estando en el rango. Si se deja vacío, se puede interactuar desde cualquier distancia.
Bomb Prefab	Bomb	Prefab de la bomba	El prefab Bomb que se instanciará en el pool. Arrastrá el prefab de la bomba desde el Project.
Spawn Point	Transform	Hijo SpawnPoint	Posición y rotación desde donde aparecen las bombas. Mover el hijo SpawnPoint en la Scene View para ajustar desde dónde caen.
Spawn Interval	Float	Default: 2	Segundos entre cada bomba spawneada mientras la palanca está ON. Valores bajos = lluvia de bombas más intensa.
Initial Delay	Float	Default: 1	Segundos que espera antes de spawnear la primera bomba al activarse. Útil para dar un margen de reacción al jugador recién activó la palanca.

🏊 Pool Settings

Campo	Tipo	Rango / Default	Descripción
Default Capacity	Int	Default: 10	Cantidad de bombas pre-instanciadas al inicio. Un valor igual o mayor al número máximo de bombas simultáneas evita creaciones en runtime.
Max Size	Int	Default: 20	Límite máximo del pool. Si se supera este número, las bombas nuevas se destruyen en lugar de devolverse al pool.

✨ Lever VFX

Al interactuar con la palanca, aparece brevemente un ícono flotante (verde para ON, rojo para OFF) que sube y se desvanece. Controlado por el hijo VFX.

Campo	Tipo	Rango / Default	Descripción
Lever VFX Renderer	SpriteRenderer	Referencia al hijo VFX	Referencia al Sprite Renderer del hijo VFX. Arrastrá el hijo VFX aquí.
VFX On Sprite	Sprite	Ícono de activación	Sprite que aparece al activar la palanca (ej: check verde, símbolo ON).
VFX Off Sprite	Sprite	Ícono de desactivación	Sprite que aparece al desactivar la palanca (ej: cruz roja, símbolo OFF).
VFX Float Distance	Float	Default: 1.2	Cuántas unidades sube el ícono antes de desvanecerse.
VFX Duration	Float	Default: 0.7	Duración en segundos de la animación completa del ícono flotante.

🔊 Audio

Campo	Tipo	Rango / Default	Descripción
Audio Source	AudioSource	Requerido para audio	Referencia al AudioSource del objeto. Necesario para reproducir los sonidos de la palanca.
Lever On Clip	AudioClip	Sonido de activación	Clip que suena al activar la palanca (ON).
Lever Off Clip	AudioClip	Sonido de desactivación	Clip que suena al desactivar la palanca (OFF).

Hijo — SpawnPoint

GameObject vacío. Su Transform define la posición y rotación de aparición de cada bomba. Moverlo en la Scene View es la forma de controlar desde dónde caen las bombas.

Campo	Tipo	Rango / Default	Descripción
Position (Transform)	Vector3	Sobre la palanca	Posición desde la que aparecen las bombas. Generalmente se coloca por encima del nivel o en un punto del techo de la sala.
Rotation (Transform)	Quaternion	Rotation (0,0,0)	Rotación de spawn. Raramente necesita cambiarse salvo que la bomba deba aparecer inclinada por diseño.

Hijo — RangeDetector

Detecta si el jugador está cerca de la palanca para habilitarle la interacción. Es un sensor de proximidad simple: solo registra si el jugador entró o salió de su zona.

🏷️ *Requiere que el jugador tenga el tag 'Player' asignado. RangeDetector usa CompareTag() internamente. Sin ese tag, la palanca nunca registrará la presencia del jugador.*

Circle Collider 2D

Campo	Tipo	Rango / Default	Descripción
Radius	Float	Default: 1.5–2	Radio de detección de proximidad en unidades. Debe ser lo suficientemente grande para que el jugador pueda activar la palanca de forma natural sin tener que estar encima de ella.
Is Trigger	Bool	✅ Activado	Debe estar activado. RangeDetector usa OnTriggerEnter2D y OnTriggerExit2D para detectar al jugador.

Script — RangeDetector

No tiene parámetros configurables en el Inspector. Expone una propiedad pública IsPlayerInRange que BombSpawner consulta al intentar interactuar.

Referencia Rápida — Problemas Comunes

Síntoma	Qué revisar
El trampolín no lanza al jugador	Verificar que Player Layer tenga la capa del jugador. El Box Collider debe estar en modo trigger (el código lo fuerza, pero confirmar que no hay conflictos con otros colliders).
La bomba no explota al tocar el suelo	Confirmar que Ground Layer tenga la capa del suelo. La bomba solo inicia la mecha al colisionar con esa capa.
La onda de choque no aparece	Verificar que Shockwave VFX tenga asignada la referencia al hijo VFX en el Inspector de Bomb.
La palanca no responde al interactuar	Asegurar que: 1) el jugador tiene el tag 'Player', 2) PlayerInteractor está activo, 3) la palanca está en la Interactable Layer configurada en PlayerInteractor.
Las bombas no se spawnean aunque la palanca está ON	Revisar que Bomb Prefab y Spawn Point estén asignados en BombSpawner. Revisar la consola por errores de pool.
El ícono VFX de la palanca no aparece	Verificar que Lever VFX Renderer esté asignado y que VFX On/Off Sprite tengan sprites asignados.

Prefabs 04 · 05 · 06 — Trampoline / Bomb / Lever · Continúa en la siguiente sección

PREFABS 07 & 08 — COIN / FLAG

Prefab 07 — Coin

La moneda es un coleccionable que otorga puntos al jugador al tocarlo. Al ser recolectado, emite un evento global de puntuación, reproduce un sonido y opcionalmente instancia un VFX, luego se destruye.

Cuadrito de dialogo para lxs manijas que quieren aprender como funciona, no hace falta a nivel curso.

> 📡 *Evento estático: OnCollected es un evento Action<int> estático en CollectibleItem. Cualquier sistema de puntuación (HUD, GameManager, etc.) puede suscribirse a él sin necesitar una referencia directa al objeto. Funciona como un broadcaster de radio que todos pueden escuchar.*

Jerarquía

```
📦 Coin                      ← GameObject raíz (trigger + lógica)
   ├── Circle Collider 2D    (isTrigger = true)
   └── CollectibleItem.cs
   │
   └── 👁️  View             ← Hijo: visual del coleccionable
        ├── Sprite Renderer
        └── SimpleFloater.cs
```

Circle Collider 2D (raíz)

Detecta cuando el jugador toca la moneda. Debe estar configurado como trigger.

Campo	Tipo	Rango / Default	Descripción
Radius	Float	Ajustar al sprite	Radio de detección. Un radio ligeramente mayor al sprite visual es más forgiving para el jugador (generous hitbox).
Is Trigger	Bool	✅ Activado	Debe estar activado. CollectibleItem usa OnTriggerEnter2D para detectar la recolección.

Script — CollectibleItem

⚙️ Configuración

Campo	Tipo	Rango / Default	Descripción
Score Value	Int	Default: 1	Puntos que se otorgan al recolectar este ítem. El valor se envía como parámetro del evento OnCollected. Monedas especiales pueden tener valores más altos (ej: moneda de oro = 5).
Collector Layer	LayerMask	Capa del jugador	Solo objetos en esta capa pueden recolectar el ítem. Asignar la capa del Player para que enemigos u otros objetos no lo recolecten accidentalmente.

💥 Feedback (Prefab)

Campo	Tipo	Rango / Default	Descripción
Pickup VFX Prefab	GameObject	Opcional	Prefab que se instancia en la posición de la moneda al ser recolectada. Idealmente contiene un OneShotEffect con AudioSource y ParticleSystem que se autodestruye. Dejar vacío para deshabilitar el VFX.

🔊 Audio

Campo	Tipo	Rango / Default	Descripción
SFX Channel	AudioEventChannelSO	Canal de SFX	Canal ScriptableObject al que se envía la petición de audio. Debe ser el mismo canal que escucha el AudioManager de la escena.
Coin SFX	AudioConfigSO	Sonido de moneda	ScriptableObject con la configuración del sonido de recolección (clip, volumen, pitch).

Hijo — View

Sprite Renderer

Renderiza el sprite de la moneda. No es controlado por ningún script; muestra el sprite estático asignado.

Campo	Tipo	Rango / Default	Descripción
Sprite	Sprite	Arte de la moneda	El gráfico de la moneda.
Sorting Layer / Order	SortingLayer / Int	Collectibles o similar	Capa de renderizado. Ajustar para que las monedas se vean por encima del fondo pero no encima del jugador.

Script — SimpleFloater

Aplica una animación de flotación sinusoidal y rotación continua al objeto. No tiene física: mueve directamente el Transform en Update. Es puramente cosmético.

🌊 *Analogía: Imaginá una boya en el mar. SimpleFloater hace exactamente eso: el objeto sube y baja suavemente como si flotara en el agua, mientras gira sobre sí mismo para llamar la atención del jugador.*

Campo	Tipo	Rango / Default	Descripción
Amplitude	Float	Default: 0.25	Altura máxima del movimiento de flotación en unidades de Unity. 0.25 = se mueve 0.25 unidades hacia arriba y 0.25 hacia abajo desde su posición inicial.
Frequency	Float	Default: 2	Velocidad del ciclo de flotación en oscilaciones por segundo. 2 = dos subidas y bajadas por segundo. Valores altos = flotación nerviosa; bajos = movimiento lento y solemne.
Rotation Speed	Float	Default: 100	Grados por segundo de rotación. 100 = una vuelta completa cada 3.6 segundos. Solo rota en el eje Y (Space.World), lo que en 2D lo hace girar como si fuera un objeto 3D.

⚠️ *SimpleFloater sobreescribe la posición Y del Transform en cada Update. Si se coloca un Rigidbody2D en el mismo objeto, habrá conflicto. Este script es solo para objetos estáticos sin física.*

Prefab 08 — Flag

La bandera es el trigger de fin de nivel. Al tocarla, el jugador queda inmovilizado y aparece un panel de fin de juego con fade-in. Es el único prefab que interactúa directamente con el sistema de UI (CanvasGroup).

Jerarquía

```
📦 Flag                    ← GameObject raíz
   ├── Sprite Renderer
   ├── Animator
   ├── Box Collider 2D      (isTrigger = true)
   └── EndFlag.cs
```

Componentes Nativos

Sprite Renderer

Muestra el sprite de la bandera. Si se usa el Animator, este campo es gestionado por las animaciones.

Campo	Tipo	Rango / Default	Descripción
Sprite	Sprite	Sprite de la bandera	Sprite estático si no se usa Animator, o el frame inicial si se usa.
Sorting Layer / Order	SortingLayer / Int	Props o similar	Capa de renderizado. Ajustar según el orden visual deseado en la escena.

Animator

Gestiona las animaciones de la bandera (ej: bandera ondeando en idle, animación de victoria al ser tocada). EndFlag no escribe ningún parámetro en el Animator, por lo que cualquier trigger de victoria debe ser gestionado manualmente o mediante un script adicional.

Campo	Tipo	Rango / Default	Descripción
Controller	AnimatorController	Requerido si se usa animación	El Animator Controller con los estados de la bandera. Puede dejarse vacío si se usa solo un sprite estático.

Box Collider 2D

Zona de detección que activa el fin de nivel. Debe ser trigger.

Campo	Tipo	Rango / Default	Descripción
Size (X, Y)	Vector2	Ajustar al área deseada	El área de activación. Puede ser más grande que el sprite visual para que sea más fácil de alcanzar, o exactamente del tamaño del poste de la bandera para mayor precisión.
Is Trigger	Bool	✅ Activado	Debe estar activado. EndFlag usa OnTriggerEnter2D para detectar al jugador.

Script — EndFlag

🏁 *Flujo al tocar la bandera: 1) Detecta colisión con tag 'Player' → 2) Detiene al jugador (desactiva input y frisa el eje X del Rigidbody) → 3) Inicia fade-in del panel de fin de juego. Solo se activa una vez gracias al flag _isTriggered.*

🖥️ UI Dependencies

Campo	Tipo	Rango / Default	Descripción
End Game Panel	CanvasGroup	REQUERIDO (error crítico)	Referencia al CanvasGroup del panel de fin de juego. Si se deja vacío, el script detiene el modo Play en el editor con un error. En build, lanzará una excepción. Es obligatorio asignarlo.
Fade Duration	Float	Default: 0.5	Segundos que tarda el panel en aparecer desde alpha 0 hasta alpha 1. Valores bajos = aparición brusca; 0.5–1.0s da una transición suave.

🔧 *Compatibilidad con IControllable: Si el PlayerController implementa la interfaz IControllable, EndFlag llama a DisableControl() para detenerlo limpiamente. Si no la implementa (caso actual del PlayerController de este proyecto), el script hace fallback y desactiva el PlayerInput y frisa el Rigidbody directamente. Ambos caminos funcionan.*

🏷️ *Requiere tag 'Player': EndFlag usa CompareTag('Player') para filtrar colisiones. El jugador debe tener asignado el tag 'Player' en su GameObject raíz, o la bandera nunca se activará.*

Referencia Rápida — Problemas Comunes

Síntoma	Qué revisar
La moneda no se recolecta al tocarla	Verificar que Collector Layer tenga la capa del Player y que el Circle Collider tenga Is Trigger activado.
No suena el audio de la moneda	Revisar que SFX Channel y Coin SFX estén asignados, y que el AudioManager de la escena escuche el mismo canal.
La bandera no activa el fin de nivel	Confirmar que el jugador tenga el tag 'Player' y que el Box Collider de la bandera tenga Is Trigger activado.
El panel de fin de juego no aparece	End Game Panel es obligatorio. Verificar que esté asignado en el Inspector. Si está vacío, el script detiene el juego en el editor.

Prefabs 07 & 08 — Coin / Flag · Continúa en la siguiente sección

Aca ya se pone más difícil a nivel teórico.

Está explicado paso a paso para no perderse.

SCRIPTABLE OBJECTS
Sistema de Audio

AudioConfigSO
Taller/Scripts/Audio/SO

1. ¿Qué es un ScriptableObject?

Un ScriptableObject es un archivo de datos que vive en la carpeta del proyecto (Project), no en la escena. A diferencia de los componentes que se adjuntan a GameObjects, los ScriptableObjects son activos reutilizables que pueden ser referenciados por múltiples objetos al mismo tiempo.

> 🗂️ *Analogía: Pensá en un ScriptableObject como una ficha técnica de un producto. En lugar de que cada objeto del juego tenga sus propios datos escritos a mano, todos consultan la misma ficha. Si cambiás el volumen en la ficha, todos los objetos que la usan se actualizan al mismo tiempo.*

En este proyecto, cada sonido del juego tiene su propio archivo AudioConfigSO en la carpeta:

```
Assets / Taller / Scripts / Audio / SO /
```

2. Inventario de AudioConfigSO

Estos son todos los assets de audio configurables actualmente en el proyecto:

Nombre del Asset	Usado por	Para qué sonido
Coin_SFX	CollectibleItem (Coin)	Sonido al recolectar una moneda
Player_Hit_SFX	PlayerAudio (Player)	Sonido al recibir daño
Player_Jump_SFX	PlayerAudio (Player)	Sonido al saltar
Player_Land_SFX	PlayerAudio (Player)	Sonido al aterrizar
UI_Click_SFX	Botones de UI (menú / HUD)	Sonido al hacer click en botones de interfaz

📂 *Cómo encontrarlos en el editor: En la ventana Project, navegá a Assets → Taller → Scripts → Audio → SO. Los archivos se muestran con el ícono de ScriptableObject (hoja con fondo azul). Hacé doble click en cualquiera para abrirlo en el Inspector.*

3. Campos del AudioConfigSO

Al seleccionar cualquier asset AudioConfigSO en el Project y verlo en el Inspector, encontrarás los siguientes campos:

Campo	Rango / Default	Descripción
Clip	AudioClip — Requerido	El archivo de audio (.wav, .mp3, .ogg) que se reproducirá. Arrastrá el clip desde la carpeta de sonidos del proyecto. Sin un Clip asignado, el SO no reproduce nada.
Mixer Group	AudioMixerGroup — Opcional	Grupo del Audio Mixer al que se envía este sonido (ej: SFX, Music, UI). Permite controlar el volumen global de cada categoría de sonidos desde el Audio Mixer. Dejar vacío reproduce el sonido sin procesamiento de mixer.
Volume	Float 0 – 1 | 1	Volumen de reproducción. 1 = volumen completo del clip; 0.5 = mitad de volumen. Ajustar aquí si un sonido específico es demasiado fuerte o demasiado suave respecto al resto.
Min Pitch	Float 0.1 – 3 | 0.95	Pitch mínimo del rango de variación. Cada vez que el sonido se reproduce, se elige un pitch aleatorio entre Min Pitch y Max Pitch. Mantener cerca de 1 para variación sutil.
Max Pitch	Float 0.1 – 3 | 1.05	Pitch máximo del rango de variación. Un rango de 0.95–1.05 es casi imperceptible pero evita que el mismo sonido suene idéntico repetido. Un rango de 0.8–1.2 da variación audible clara.
Loop	Bool | false	Si el sonido debe repetirse en loop. Para SFX de acción (salto, moneda, golpe) debe estar en false. Activar solo para sonidos ambientales o música de fondo gestionada por este sistema.

4. Cómo cambiar un sonido

Paso a paso para reemplazar o ajustar cualquier sonido del juego:

📁 Paso 1 — Importar el nuevo audio al proyecto

Arrastrá el archivo de audio (.wav o .ogg recomendado para SFX) a la carpeta de sonidos del proyecto en la ventana Project. Una vez importado, aparecerá como un asset AudioClip con el ícono de onda de sonido.

🎧 *Formatos recomendados: .wav o .aiff para SFX cortos (mejor calidad sin compresión extra). .mp3 u .ogg para música (compresión aceptable para clips largos). Unity importa y convierte automáticamente; no es necesario pre-convertir.*

🔍 Paso 2 — Abrir el AudioConfigSO

En la ventana Project, navegá a Assets → Taller → Scripts → Audio → SO y hacé click en el asset que querés modificar (ej: Player_Jump_SFX). Se abrirá en el Inspector.

🔄 Paso 3 — Asignar el nuevo clip

En el campo Clip del Inspector, arrastrá el nuevo AudioClip importado en el Paso 1, o hacé click en el selector circular (⊙) a la derecha del campo para abrir el buscador de assets.

🔊 Paso 4 — Ajustar volumen y pitch (opcional)

Si el nuevo clip suena demasiado fuerte o suave respecto al resto del juego, ajustá el campo Volume. Si querés más o menos variación de tono entre reproducciones, modificá el rango Min Pitch / Max Pitch.

▶️ Paso 5 — Probar en Play Mode

Entrá en Play Mode y activá la acción correspondiente en el juego para escuchar el nuevo sonido en contexto. Los cambios en ScriptableObjects se aplican inmediatamente sin necesidad de recompilar.

5. Cómo crear un nuevo AudioConfigSO

Si necesitás agregar un sonido nuevo al juego (para un nuevo prefab, un nuevo evento, etc.) podés crear tu propio asset:

1	En la ventana Project, hacé click derecho sobre la carpeta SO (o cualquier carpeta de tu elección).
2	En el menú contextual, seleccioná: Create → Audio → Audio Config
3	Se creará un nuevo asset llamado 'NewAudioConfig'. Renombralo según el sonido (ej: Enemy_Death_SFX).
4	Seleccioná el nuevo asset y asigná los campos en el Inspector (Clip, Volume, Pitch, etc.).
5	Arrastrá el nuevo asset al campo correspondiente del script que lo necesita (ej: el campo Coin SFX de CollectibleItem, o el campo Jump SFX de PlayerAudio).

6. Referencia Rápida — Problemas Comunes

Síntoma	Qué revisar
El campo Clip está vacío en el SO	Asignar un AudioClip en el campo Clip del Inspector. Sin un clip, el SO no hace nada al llamar a Play().
El sonido suena demasiado fuerte/suave	Ajustar el campo Volume del AudioConfigSO correspondiente. No modificar el volumen del AudioSource directamente.
El sonido no varía entre reproducciones	Ampliar el rango entre Min Pitch y Max Pitch (ej: 0.85 – 1.15). Con Min = Max el pitch siempre es idéntico.

ScriptableObjects de Audio — AudioConfigSO · Continúa en la siguiente sección

1. CameraShake

CameraShake es una extensión de Cinemachine que agrega temblor de cámara basado en la distancia a una explosión. Funciona como singleton: hay una sola instancia en la escena y cualquier script puede llamarla con CameraShake.Instance.

📷 *Analogía: Imaginá que la cámara es una copa de agua sobre una mesa. Cuando explota una bomba cerca, el agua tiembla fuerte; si explota lejos, apenas se nota. CameraShake hace exactamente eso: cuanto más cerca está la explosión de la cámara, más fuerte es el temblor.*

Ubicación en la jerarquía

CameraShake se adjunta como componente adicional a la CinemachineCamera virtual de la escena. No es un prefab independiente; es parte del sistema de cámara.

```
📦 Scene Hierarchy
   └── 🎥 CM_VirtualCamera
        ├── CinemachineCamera     ← componente nativo de Cinemachine
        ├── CameraShake.cs        ← extensión: temblor de cámara
        └── CameraShakeListener.cs ← expone Position para distancia
```

⚠️ *Requerimiento crítico: La CinemachineCamera debe tener asignado un Tracking Target (el jugador u otro objeto). Si el campo Follow está vacío, el script detiene el Play Mode con un error en el editor.*

Parámetros del Inspector

Campo	Tipo	Rango / Default	Descripción
Default Duration	Float	Default: 0.3	Duración del temblor en segundos cuando se llama ShakeFromPosition() sin especificar duración. Bomb.cs pasa sus propios valores; este default se usa si algún sistema llama al shake sin argumentos.

Campo	Tipo	Rango / Default	Descripción
Default Magnitude	Float	Default: 0.25	Intensidad máxima del temblor cuando se llama sin especificar magnitud. A mayor valor, más desplazamiento de cámara.
Noise Frequency	Float	Default: 40	Velocidad del ruido de Perlin que genera el movimiento. Valores altos = temblor más rápido y caótico; valores bajos = balanceo lento. 40 da sensación de explosión; 5–10 daría sensación de terremoto.
Max Shake Distance	Float	Default: 15	Radio máximo en unidades de Unity dentro del cual una explosión genera temblor. Explosiones fuera de este radio son ignoradas completamente. Ajustar al tamaño de las salas del nivel.

📐 *Cómo funciona la atenuación por distancia: El temblor no es uniforme dentro del radio. Se usa una curva cuadrática (ratio²): a mitad del radio la cámara tiembla solo al 25% de la intensidad máxima. Cuanto más cerca de la explosión, exponencialmente más fuerte. Esto hace que sea muy notorio cerca y prácticamente imperceptible lejos.*

2. Sistema de Capas (Layers)

Unity usa Layers para controlar qué objetos interactúan entre sí en física y detección. Todos los Tilemaps y GameObjects tienen asignada una Layer. Los scripts como PlayerController, SimplePatrolEnemy y CollectibleItem usan LayerMask para filtrar con qué capas quieren interactuar.

> 🏗️ *Analogía: Pensá en los Layers como departamentos de un edificio. Cada objeto vive en un piso. Los scripts pueden llamar por teléfono solo a los pisos que les interesan (usando LayerMask), ignorando todos los demás. Dos objetos en pisos diferentes pueden superponerse visualmente sin colisionar físicamente.*

2.1 Resumen de capas del proyecto

Layer	Nombre en Unity	Objetos que la usan
🔵	**Ground**	Tilemap Grass — suelo sólido principal
🟢	**OneWayPlatforms**	Tilemap OneWayPlatforms — plataformas semisólidas
🔴	**Damaged**	Tilemap Danger — zonas de daño (pinchos, lava, etc.)
🟣	**Default**	Tilemap Decorations — elementos puramente visuales, sin colisión
🟡	**Ladder**	Tilemap Ladders — zonas de escalada
🟢	**Player**	Prefab Player
🟠	**Enemy (sugerida)**	Prefabs Bee, Slime — enemigos de patrulla

2.2 Detalle por capa

• Ground — Tilemap: Grass

El suelo sólido principal del nivel. Los jugadores y enemigos no lo atraviesan en ninguna dirección. Es la capa de referencia para la detección de suelo del PlayerController y para que la Bomb inicie su mecha al tocar.

Tilemap / Objeto	Interactúa con	Comportamiento
Grass (Tilemap)	PlayerController — What Is Ground	El jugador detecta que está en suelo y puede saltar. Activa el evento OnLandPerformed al aterrizar.
Grass (Tilemap)	PlayerController — What Is Ground	Coyote Time y Jump Buffer se resetan al aterrizar sobre esta capa.
Grass (Tilemap)	Bomb — Ground Layer	Al colisionar con esta capa, la bomba inicia su mecha y el contador de detonación comienza.
Grass (Tilemap)	SimplePatrolEnemy (Kinematic)	Los enemigos se mueven sobre esta superficie. Al ser Kinematic, no colisionan físicamente, pero la geometría del tilemap sirve como referencia visual de sus rutas.

• OneWayPlatforms — Tilemap: OneWayPlatforms

Plataformas que el jugador puede atravesar hacia arriba (saltando desde abajo) y también bajar a través de ellas presionando Abajo + Salto. Los enemigos no interactúan con esta capa de forma especial.

Tilemap / Objeto	Interactúa con	Comportamiento
OneWayPlatforms (Tilemap)	PlayerController — One Way Platform Layer	El jugador puede aterrizar encima y bajar a través con Abajo + Salto (Drop Down). El collider de la plataforma se ignora temporalmente durante el Drop Down Duration.
OneWayPlatforms (Tilemap)	PlayerController — What Is Ground	Esta capa también se suma a What Is Ground para que el jugador pueda saltar desde encima. Durante el Drop Down, se excluye de What Is Ground temporalmente.

⚙️ *Cómo funciona técnicamente: Al bajar, PlayerController usa Physics2D.IgnoreCollision() para deshabilitar la colisión entre el Capsule Collider del jugador y cada plataforma del área. Tras Drop Down Duration segundos, la colisión se restaura automáticamente.*

● Damaged — Tilemap: Danger

Zonas que dañan al jugador al contacto. Pinchos, lava, espinas o cualquier elemento de escenario que cause daño se coloca en este tilemap. El daño funciona por colisión/trigger, no por código del tilemap en sí.

Tilemap / Objeto	Interactúa con	Comportamiento
Danger (Tilemap)	PlayerController — Damage Layer	Al colisionar o solaparse con cualquier objeto en esta capa, el jugador recibe daño con knockback y hitStun. El Damage Cooldown evita recibir daño múltiple consecutivo.
Danger (Tilemap)	SimplePatrolEnemy	Los enemigos NO interactúan con esta capa. Si un enemigo pasa por encima de pinchos, no recibe daño (solo el jugador tiene la lógica de TakeDamage).

⚠️ *Para que un Tilemap dañe al jugador, su Collider (TilemapCollider2D) debe estar configurado como trigger O el PlayerController debe detectarlo por colisión normal. Verificar la configuración del TilemapCollider2D si el daño no funciona.*

● Default — Tilemap: Decorations

Elementos puramente visuales del nivel: fondos, detalles decorativos, plantas, nubes. Esta capa no participa en ninguna LayerMask del proyecto, por lo que ningún script la detecta ni colisiona con ella.

Tilemap / Objeto	Interactúa con	Comportamiento
Decorations (Tilemap)	Ningún script	Sin colisión física ni detección. Es ignorado por todos los sistemas del juego. Solo afecta el renderizado visual.

💡 *El Tilemap Decorations no necesita TilemapCollider2D. Si tiene uno asignado accidentalmente, puede generar colisiones inesperadas. Verificar que no tenga collider activo.*

● Ladder — Tilemap: Ladders

Zonas de escalada. El jugador puede agarrarse a estas áreas y moverse verticalmente. La escalera es un trigger, no un obstáculo físico: el jugador la atraviesa libremente y solo activa el modo escalada al presionar Arriba o Abajo estando en contacto.

Tilemap / Objeto	Interactúa con	Comportamiento
Ladders (Tilemap)	PlayerController — Ladder Layer	Al tocar esta capa y presionar Arriba (desde el suelo) o cualquier dirección vertical (en el aire), el jugador entra en modo escalada: se desactiva la gravedad y el movimiento vertical es controlado directamente.
Ladders (Tilemap)	PlayerController — Ladder Layer	Al llegar al suelo mientras escala y presionar Abajo, el jugador sale de la escalera. Al saltar desde la escalera, sale con impulso normal de salto.

🪜 *El Tilemap Ladders debe tener un TilemapCollider2D configurado como trigger (Is Trigger = true). Si está como colisión sólida, el jugador no podrá atravesarlo y quedará bloqueado. El script detecta contacto con IsTouchingLayers(), que funciona con triggers.*

2.3 Matriz de interacciones

Esta tabla resume qué LayerMask usa cada script y con qué capa debe configurarse para que el juego funcione correctamente:

Script	Campo LayerMask	Layer a asignar	Qué detecta
PlayerController	What Is Ground	**Ground + OneWayPlatforms**	*Suelo para salto, coyote time y aterrizaje*
PlayerController	One Way Platform Layer	**OneWayPlatforms**	*Plataformas por las que puede bajar*
PlayerController	Ladder Layer	**Ladder**	*Zonas de escalada*
PlayerController	Damage Layer	**Damaged**	*Objetos que dañan al jugador*
CollectibleItem	Collector Layer	**Player**	*Quién puede recolectar la moneda*
Trampoline	Player Layer	**Player**	*Quién activa el rebote*
Bomb	Ground Layer	**Ground + OneWayPlatforms**	*Contacto que inicia la mecha*
PlayerInteractor	Interactable Layer	**Interactable (custom)**	*Objetos con IInteractable en rango*
RangeDetector	Tag: Player	**— (usa tag, no layer)**	*Presencia del jugador cerca de la palanca*

3. Referencia Rápida — Problemas Comunes

Síntoma	Qué revisar
La cámara no tiembla al explotar una bomba	Verificar que: 1) CameraShake esté adjunto a la CinemachineCamera, 2) la distancia entre la bomba y la cámara sea menor que Max Shake Distance, 3) el CameraShakeListener esté en la escena.
La cámara tiembla pero muy poco	Aumentar Default Magnitude en CameraShake o Shake Magnitude en Bomb. Verificar que la bomba explota cerca de la cámara (dentro del Max Shake Distance).
El jugador no puede bajar por plataformas	Verificar que el Tilemap OneWayPlatforms tenga la layer OneWayPlatforms y que One Way Platform Layer en PlayerController esté configurada.
El jugador no recibe daño de los pinchos	Confirmar que el Tilemap Danger tenga la layer Damaged y que Damage Layer en PlayerController incluya esa capa. El collider del tilemap debe estar como trigger O como colisión sólida según cómo esté configurado el TryTakeDamage.

Síntoma	Qué revisar
El jugador no puede trepar la escalera	Verificar que el Tilemap Ladders tenga la layer Ladder, que Ladder Layer en PlayerController esté configurada, y que el TilemapCollider2D de Ladders tenga Is Trigger = true.
Las decoraciones tienen colisión accidental	El Tilemap Decorations no debe tener TilemapCollider2D activo. Remover o deshabilitar el componente collider del tilemap de decoraciones.

Sistemas de Escena — CameraShake & Layers · Fin del documento